

DA5020 PRACTICUM 1

Members

Deepali Krishna Kashyap, Jan Mollet, Kanchan Sadashiv Deore, Nidhi Nityanand Uchil, Shamitha K V,

2025-02-16

Loading Libraries

```
library(tidyverse)
```

```
## Warning: package 'dplyr' was built under R version 4.4.3
```

```
## -- Attaching core tidyverse packages ----- tidyverse 2.0.0 --
```

```
## v dplyr      1.1.4      v readr      2.1.5
```

```
## v forcats    1.0.0      v stringr    1.5.1
```

```
## v ggplot2    3.5.1      v tibble     3.2.1
```

```
## v lubridate  1.9.4      v tidyr      1.3.1
```

```
## v purrr      1.0.2
```

```
## -- Conflicts ----- tidyverse_conflicts() --
```

```
## x dplyr::filter() masks stats::filter()
```

```
## x dplyr::lag()     masks stats::lag()
```

```
## i Use the conflicted package (<http://conflicted.r-lib.org/>) to force all conflicts to become errors
```

```
library(lubridate)
```

```
library(XML)
```

```
library(dplyr)
```

```
library(stringr)
```

```
library(tidyr)
```

```
library(ggplot2)
```

PART 1

Question 1.

Create a dataframe with 4 variables. The 4 variables should be doctor_type, doctor_lastname, location, AVG_Rating .

The variable doctor_type should contain 4 inputs (PCP, Psychiatrist, Surgeon, Anesthesia) The variable doctor_lastname should contain 4 inputs (Smith, Dame, Jones, Zayas) The variable location should contain 4 inputs (MA, ME, NH, VT) The variable AVG_Rating should contain 4 inputs (7,9,8,9) Print the data frame.

```
doctor_type <- c("PCP","Psychiatrist", "Surgeon", "Anesthesia")
```

```
doctor_lastname <- c("Smith", "Dame", "Jones", "Zayas")
```

```
location<- c("MA", "ME", "NH", "VT")
```

```
AVG_Rating <- c(7, 9, 8, 9)
```

```
doctor_df <-data.frame(doctor_type, doctor_lastname, location, AVG_Rating)

doctor_df
```

```
##      doctor_type doctor_lastname location AVG_Rating
## 1          PCP          Smith      MA          7
## 2 Psychiatrist          Dame      ME          9
## 3        Surgeon          Jones      NH          8
## 4    Anesthesia          Zayas      VT          9
```

The code creates a data frame (doctor_df) that stores information about different types of doctors, their last names, locations, and average ratings. It then prints the data frame.

Question 2.

Using the dataframe above. Select row 1 in column 2, what was selected? Select rows 2 through 4, what was selected? Select the last column, what was selected?

```
#select row 1 in column 2
doctor_df[1,2]
```

```
## [1] "Smith"
```

To select rows and columns within a Data Frame, we use square brackets. The first argument indicates which row we want to select. In this case, row = 1. The second argument selects the column. In this case, column = 2. For the first iteration, we are retrieving the last name of the first doctor stored, which is 'Smith'.

```
#select rows 2 through 4
doctor_df[2:4,]
```

```
##      doctor_type doctor_lastname location AVG_Rating
## 2 Psychiatrist          Dame      ME          9
## 3        Surgeon          Jones      NH          8
## 4    Anesthesia          Zayas      VT          9
```

In this case, when we use ':', we are specifying a range. Since this range is in the rows section, we are selecting the rows from 2 to 4 inclusively. In other words, since our Data Frame has only four rows, we are selecting the entire Data Frame except for the first row. The column argument remains empty, indicating that we do not want to select any specific column. We are retrieving all the data for the last three doctors in our Data Frame, including their specialties, last names, locations, and ratings.

```
#Last column
doctor_df[,4]
```

```
## [1] 7 9 8 9
```

Finally, we are selecting only the column located in the fourth position within the Data Frame. We are retrieving only the AVG_ratings for all the doctors.

Question 3.

Using the dataset Mtcars create a scatterplot showing the relations between any two variables of your choosing. Explain why you picked those variables. Each point should also be based on a color scheme of your choosing.

```
head(mtcars)
```

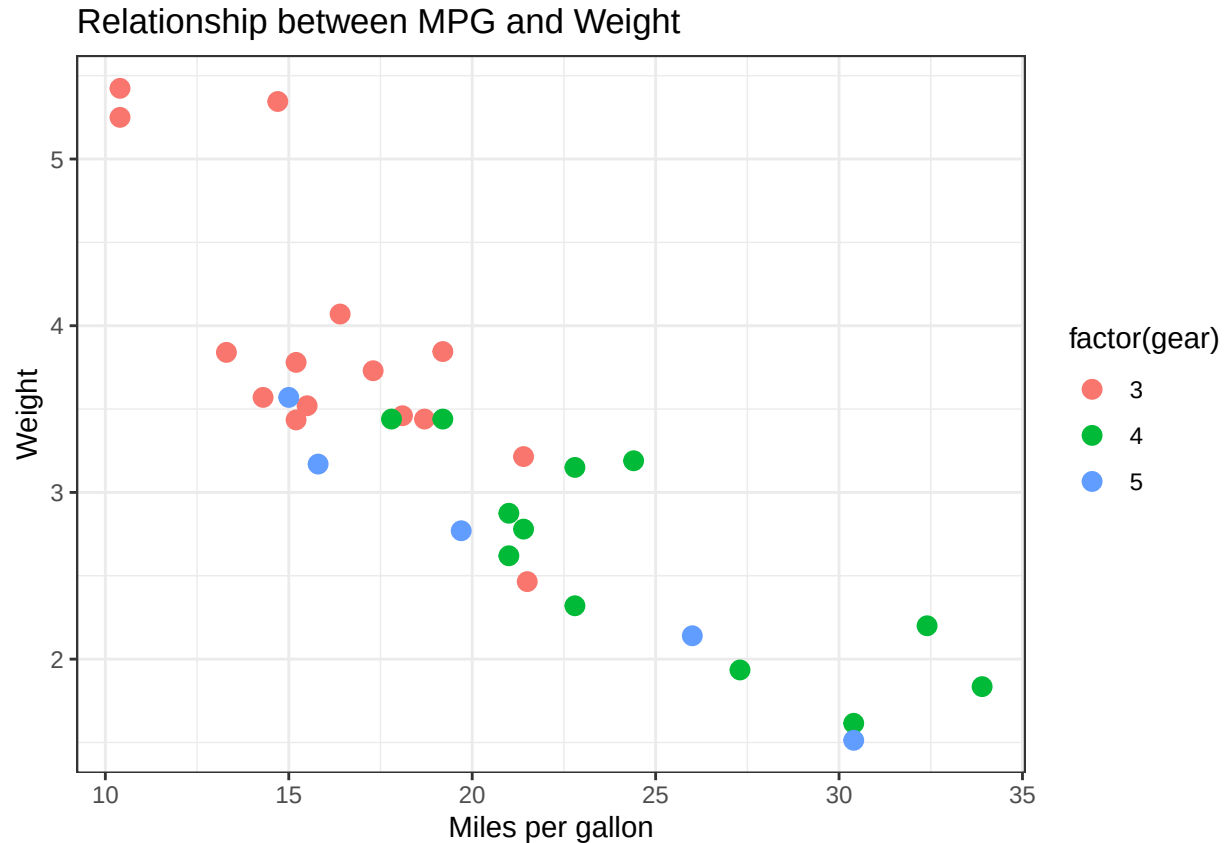
```
##           mpg cyl disp  hp drat   wt  qsec vs am gear carb
## Mazda RX4      21.0   6  160 110 3.90 2.620 16.46  0  1    4    4
## Mazda RX4 Wag  21.0   6  160 110 3.90 2.875 17.02  0  1    4    4
## Datsun 710      22.8   4  108  93 3.85 2.320 18.61  1  1    4    1
## Hornet 4 Drive  21.4   6  258 110 3.08 3.215 19.44  1  0    3    1
## Hornet Sportabout 18.7   8  360 175 3.15 3.440 17.02  0  0    3    2
## Valiant         18.1   6  225 105 2.76 3.460 20.22  1  0    3    1
```

```
str(mtcars)
```

```
## 'data.frame':   32 obs. of  11 variables:
##  $ mpg : num  21 21 22.8 21.4 18.7 18.1 14.3 24.4 22.8 19.2 ...
##  $ cyl : num  6 6 4 6 8 6 8 4 4 6 ...
##  $ disp: num  160 160 108 258 360 ...
##  $ hp  : num  110 110 93 110 175 105 245 62 95 123 ...
##  $ drat: num  3.9 3.9 3.85 3.08 3.15 2.76 3.21 3.69 3.92 3.92 ...
##  $ wt  : num  2.62 2.88 2.32 3.21 3.44 ...
##  $ qsec: num  16.5 17 18.6 19.4 17 ...
##  $ vs  : num  0 0 1 1 0 1 0 1 1 1 ...
##  $ am  : num  1 1 1 0 0 0 0 0 0 0 ...
##  $ gear: num  4 4 4 3 3 3 3 4 4 4 ...
##  $ carb: num  4 4 1 1 2 1 4 2 2 4 ...
```

We display the first rows of the dataset (head function) to examine its structure and the different variables (columns) that we have. By using the str() function, we can see the dimensions of the dataset (32 observations = 32 rows; 11 variables = 11 columns), as well as the types of variables. In this case, all of the columns are numeric.

```
# Set up plot with mpg on x-axis and wt on y-axis
ggplot(mtcars, aes(x = mpg, y = wt)) +
  geom_point(aes(colour = factor(gear)), size = 3) +
  xlab("Miles per gallon") +
  ylab("Weight") +
  ggtitle("Relationship between MPG and Weight") +
  theme_bw()
```



The scatter plot visualizes the relationship between Miles per Gallon (MPG) and Weight of cars, with data points color-coded based on the number of gears (3, 4, or 5). There is a negative correlation between MPG and Weight—heavier cars tend to have lower MPG. Cars with 3 gears tend to be heavier and have lower MPG. Cars with 5 gears are generally lighter and have higher MPG.

Question 4.

Using the dataset MTCars perform a summary statistic on the dataset and calculate the pearson coefficient of the correlation R picking two variables of choice. Explain why you picked those variables and explain the purpose of a pearson coefficient.

```
# Summary of the mtcars dataset
```

```
summary(mtcars)
```

```
##      mpg          cyl        disp         hp
##  Min.   :10.40   Min.   :4.000   Min.    : 71.1   Min.    : 52.0
##  1st Qu.:15.43   1st Qu.:4.000   1st Qu.:120.8   1st Qu.: 96.5
##  Median :19.20   Median :6.000   Median :196.3   Median :123.0
##  Mean   :20.09   Mean   :6.188   Mean   :230.7   Mean   :146.7
##  3rd Qu.:22.80   3rd Qu.:8.000   3rd Qu.:326.0   3rd Qu.:180.0
##  Max.   :33.90   Max.   :8.000   Max.   :472.0   Max.   :335.0
##      drat         wt         qsec         vs
##  Min.   :2.760   Min.   :1.513   Min.    :14.50   Min.    :0.0000
##  1st Qu.:3.080   1st Qu.:2.581   1st Qu.:16.89   1st Qu.:0.0000
##  Median :3.695   Median :3.325   Median :17.71   Median :0.0000
##  Mean   :3.597   Mean   :3.217   Mean   :17.85   Mean   :0.4375
```

```
## 3rd Qu.:3.920 3rd Qu.:3.610 3rd Qu.:18.90 3rd Qu.:1.0000
## Max. :4.930 Max. :5.424 Max. :22.90 Max. :1.0000
##      am      gear      carb
## Min. :0.0000 Min. :3.000 Min. :1.000
## 1st Qu.:0.0000 1st Qu.:3.000 1st Qu.:2.000
## Median :0.0000 Median :4.000 Median :2.000
## Mean :0.4062 Mean :3.688 Mean :2.812
## 3rd Qu.:1.0000 3rd Qu.:4.000 3rd Qu.:4.000
## Max. :1.0000 Max. :5.000 Max. :8.000
```

```
# Correlation between two variables (mpg vs wt)

cor(log10(mtcars$mpg),mtcars$wt,use = "complete.obs")
```

```
## [1] -0.8930611
```

Since correlation of mpg vs wt = -0.89, there is a strong negative correlation between MPG and Weight. Just because MPG and weight have a strong negative correlation(-0.89) doesn't mean that weight causes lower fuel efficiency. There could be other factors at play. Correlation coefficient only measures linear association, not causation.

PART 2

Now that we know more about the Chemical Dependence Treatment Program offered by The Office of Addiction Services and Support, we have a better picture of the mission of The Office and their history and requirements to be accepted. From now on, we can understand the data by reading the dictionary file and loading the ".csv" file in our code chunk to observe the structure and dimensions of the dataset.

Question 1.

Load the data, directly from the URL Links to an external site, into your R environment.

```
ny_cd_df <- read.csv("https://query.data.world/s/or7vayxgdtwvfceewvazzo5psj6wn?dws=00000", header=TRUE)
head(ny_cd_df)
```

```
##   Year County.of.Program.Location Program.Category
## 1 2017              Albany              Crisis
## 2 2017              Albany              Crisis
## 3 2017              Albany              Crisis
## 4 2017              Albany              Crisis
## 5 2017              Albany              Crisis
## 6 2017              Albany              Crisis
##                Service.Type Age.Group Primary.Substance.Group
## 1 Medically Managed Detoxification 18 thru 24              Alcohol
## 2 Medically Managed Detoxification 18 thru 24              All Others
## 3 Medically Managed Detoxification 18 thru 24 Cocaine incl Crack
## 4 Medically Managed Detoxification 18 thru 24              Heroin
## 5 Medically Managed Detoxification 18 thru 24              Other Opioids
## 6 Medically Managed Detoxification 25 thru 34              Alcohol
## Admissions
## 1      25
## 2       7
## 3       1
## 4      64
## 5      20
## 6     140
```

```
# We get the total columns and rows
dim(ny_cd_df)
```

```
## [1] 72463      7
```

```
# We get the structure of the data frame and the types of variables.
str(ny_cd_df)
```

```
## 'data.frame': 72463 obs. of 7 variables:
## $ Year : int 2017 2017 2017 2017 2017 2017 2017 2017 2017 ...
## $ County.of.Program.Location: chr "Albany" "Albany" "Albany" "Albany" ...
## $ Program.Category : chr "Crisis" "Crisis" "Crisis" "Crisis" ...
## $ Service.Type : chr "Medically Managed Detoxification" "Medically Managed Detoxification" ...
## $ Age.Group : chr "18 thru 24" "18 thru 24" "18 thru 24" "18 thru 24" ...
## $ Primary.Substance.Group : chr "Alcohol" "All Others" "Cocaine incl Crack" "Heroin" ...
## $ Admissions : int 25 7 1 64 20 140 10 4 244 63 ...
```

Question 2.

Evaluate the dataset to determine what data preparation steps are needed and perform them. At a minimum, ensure that you discuss the distribution of the data, outliers and prepare any helpful summary statistics to support your analysis.

We will be dealing with a data frame that has around 72,463 observations and 7 variables. The data types in our Data Frame are characters (strings) and integers (integer numbers). We can also use the `glimpse()` function to see more clearly the data type and some of the values.

```
glimpse(ny_cd_df)
```

```
## Rows: 72,463
## Columns: 7
## $ Year <int> 2017, 2017, 2017, 2017, 2017, 2017, 2017, 2017, 2~
## $ County.of.Program.Location <chr> "Albany", "Albany", "Albany", "Albany", "Al~
## $ Program.Category <chr> "Crisis", "Crisis", "Crisis", "Crisis", "Cr~
## $ Service.Type <chr> "Medically Managed Detoxification", "Medica~
## $ Age.Group <chr> "18 thru 24", "18 thru 24", "18 thru 24", "18~
## $ Primary.Substance.Group <chr> "Alcohol", "All Others", "Cocaine incl Crac~
## $ Admissions <int> 25, 7, 1, 64, 20, 140, 10, 4, 244, 63, 230,~
```

We also want to obtain a better understanding of the statistics of the Data Frame.

```
# We can get the statistics from the data frame
```

```
summary(ny_cd_df)
```

```
##      Year      County.of.Program.Location Program.Category
## Min.   :2007   Length:72463                Length:72463
## 1st Qu.:2009   Class :character              Class :character
## Median :2012   Mode  :character              Mode  :character
## Mean   :2012
## 3rd Qu.:2015
## Max.   :2017
```

```
## Service.Type      Age.Group      Primary.Substance.Group
## Length:72463      Length:72463      Length:72463
## Class :character  Class :character  Class :character
## Mode :character   Mode :character   Mode :character
##
##
##
## Admissions
## Min.   : 1.00
## 1st Qu.: 3.00
## Median : 8.00
## Mean   : 44.62
## 3rd Qu.: 30.00
## Max.   :2862.00
```

Now that we have a better understanding of the data, let's start retrieving more specific information and identifying things we may need to change for a better data wrangling process.

1. We will make arrangements to change the names of the columns for a better understanding.
2. We will change the data type of the columns that don't have the most appropriate type. For example, some of the columns are stored as characters when they should be factors because they are representing different categories.
3. Make sure our Data Frame doesn't contain NA or empty values. If it does, we will treat them separately and take the appropriate approach. In other words, should we delete them? Should we convert the NAs or empty values to 0?
4. We will address the outliers and consider the definition of an outlier. We will need to decide where the boundary of an outlier lies: Is it 3 standard deviations from the mean? Or 1.5 standard deviations from the mean?
5. Finally, we will have a clean Data Frame ready to use and extract information from.

We will ensure that the results make sense, and if for some reason we detect an error or irregularity, we will address it and restart the cleaning process

STEP 1

```
# Renaming the columns and applying the changes in the data frame. Instead of using "." we prefer to use underscores
ny_cd_df <- ny_cd_df %>%
  rename("County" = "County.of.Program.Location",
         "Service_Type" = "Service.Type",
         "Program_Category" = "Program.Category",
         "Age_Group" = "Age.Group",
         "Primary_Substance_Group" = "Primary.Substance.Group")

# Observing the change of names
head(ny_cd_df)
```

```
##   Year County Program_Category          Service_Type Age_Group
## 1 2017 Albany          Crisis Medically Managed Detoxification 18 thru 24
## 2 2017 Albany          Crisis Medically Managed Detoxification 18 thru 24
## 3 2017 Albany          Crisis Medically Managed Detoxification 18 thru 24
## 4 2017 Albany          Crisis Medically Managed Detoxification 18 thru 24
## 5 2017 Albany          Crisis Medically Managed Detoxification 18 thru 24
## 6 2017 Albany          Crisis Medically Managed Detoxification 25 thru 34
##   Primary_Substance_Group Admissions
## 1             Alcohol          25
## 2             All Others          7
## 3   Cocaine incl Crack          1
## 4             Heroin          64
## 5             Other Opioids       20
## 6             Alcohol          140
```

```
# We could also just display the name of the columns
colnames(ny_cd_df)
```

```
## [1] "Year"          "County"
## [3] "Program_Category" "Service_Type"
## [5] "Age_Group"      "Primary_Substance_Group"
## [7] "Admissions"
```

STEP 2

We will detect which columns need to be stored in a different type:

1. **“Program_Category”**: is stored as a character but because it refers to a category it should be stored as a factor.
2. **“Service_Type”**: is stored as a character but because it refers to a category it should be stored as a factor.
3. **“Age_Group”**: is stored as a character but because it refers to a category it should be stored as a factor.
4. **“Primary_Substance_Group”**: is stored as a character but because it refers to a category it should be stored as a factor.
5. **“Year”**: is stored as an integer but because it refers to a category (group of years) it should be stored as a factor.

The variables: County_of_Program_Location and Admissions will remain the same since their data types are accurate

```
ny_cd_df$Program_Category <- as.factor(ny_cd_df$Program_Category)
ny_cd_df$Service_Type <- as.factor(ny_cd_df$Service_Type)
ny_cd_df$Age_Group <- as.factor(ny_cd_df$Age_Group)
ny_cd_df$Primary_Substance_Group <- as.factor(ny_cd_df$Primary_Substance_Group)
ny_cd_df$Year <- as.factor(ny_cd_df$Year)

head(ny_cd_df)
```

```
##   Year County Program_Category          Service_Type Age_Group
## 1 2017 Albany          Crisis Medically Managed Detoxification 18 thru 24
```



```
## 2 2017 Albany Crisis Medically Managed Detoxification 18 thru 24
## 3 2017 Albany Crisis Medically Managed Detoxification 18 thru 24
## 4 2017 Albany Crisis Medically Managed Detoxification 18 thru 24
## 5 2017 Albany Crisis Medically Managed Detoxification 18 thru 24
## 6 2017 Albany Crisis Medically Managed Detoxification 25 thru 34
## Primary_Substance_Group Admissions
## 1 Alcohol 25
## 2 All Others 7
## 3 Cocaine incl Crack 1
## 4 Heroin 64
## 5 Other Opioids 20
## 6 Alcohol 140
```

STEP 3

#By using the sum() function, we can count how many NA's are present in our Data Frame. Additionally, w

```
sum(is.na(ny_cd_df))
```

```
## [1] 0
```

```
empty <- sum(ny_cd_df == "" | ny_cd_df == " ")
```

Printing the results

```
print(paste("Number of empty values:", empty))
```

```
## [1] "Number of empty values: 0"
```

Count duplicate rows

```
num_duplicates <- sum(duplicated(ny_cd_df))
```

Remove duplicates if found

```
ny_cd_df <- ny_cd_df %>% distinct()
```

Print the number of removed duplicate rows

```
print(paste("Number of duplicate rows removed:", num_duplicates))
```

```
## [1] "Number of duplicate rows removed: 0"
```

STEP 4

#ADMISSIONS

```
admissions <- ny_cd_df$Admissions
mean_admissions = mean(admissions)
```

#Standard deviations

```
sd_admissions = sd(admissions)
sd_admissions
```

```
## [1] 131.1749
```

```
#Getting the high outliers and low outliers for the variable admissions.
```

```
high_outliers_admissions = mean_admissions + 3 * sd_admissions  
high_outliers_admissions
```

```
## [1] 438.1451
```

```
low_outliers_admissions = mean_admissions - 3 * sd_admissions  
low_outliers_admissions
```

```
## [1] -348.9043
```

```
#Getting the number of outliers considering the values below the low_outliers and above the high_outliers  
n_admissions_outliers <- sum(admissions < low_outliers_admissions | admissions > high_outliers_admissions)
```

```
#We've got 1380 outliers for "admissions"  
n_admissions_outliers
```

```
## [1] 1380
```

```
# We want now to eliminate the outliers from the column tip_amount. We will use a filter to cleanup the  
ny_cd_df_clean <- ny_cd_df %>%  
  filter(admissions >= low_outliers_admissions & admissions <= high_outliers_admissions)
```

```
#Comparing both data frames:  
head(ny_cd_df_clean)
```

```
##   Year County Program_Category      Service_Type Age_Group  
## 1 2017 Albany      Crisis Medically Managed Detoxification 18 thru 24  
## 2 2017 Albany      Crisis Medically Managed Detoxification 18 thru 24  
## 3 2017 Albany      Crisis Medically Managed Detoxification 18 thru 24  
## 4 2017 Albany      Crisis Medically Managed Detoxification 18 thru 24  
## 5 2017 Albany      Crisis Medically Managed Detoxification 18 thru 24  
## 6 2017 Albany      Crisis Medically Managed Detoxification 25 thru 34  
##   Primary_Substance_Group Admissions  
## 1           Alcohol          25  
## 2           All Others           7  
## 3   Cocaine incl Crack           1  
## 4           Heroin          64  
## 5       Other Opioids          20  
## 6           Alcohol         140
```

```
head(ny_cd_df)
```

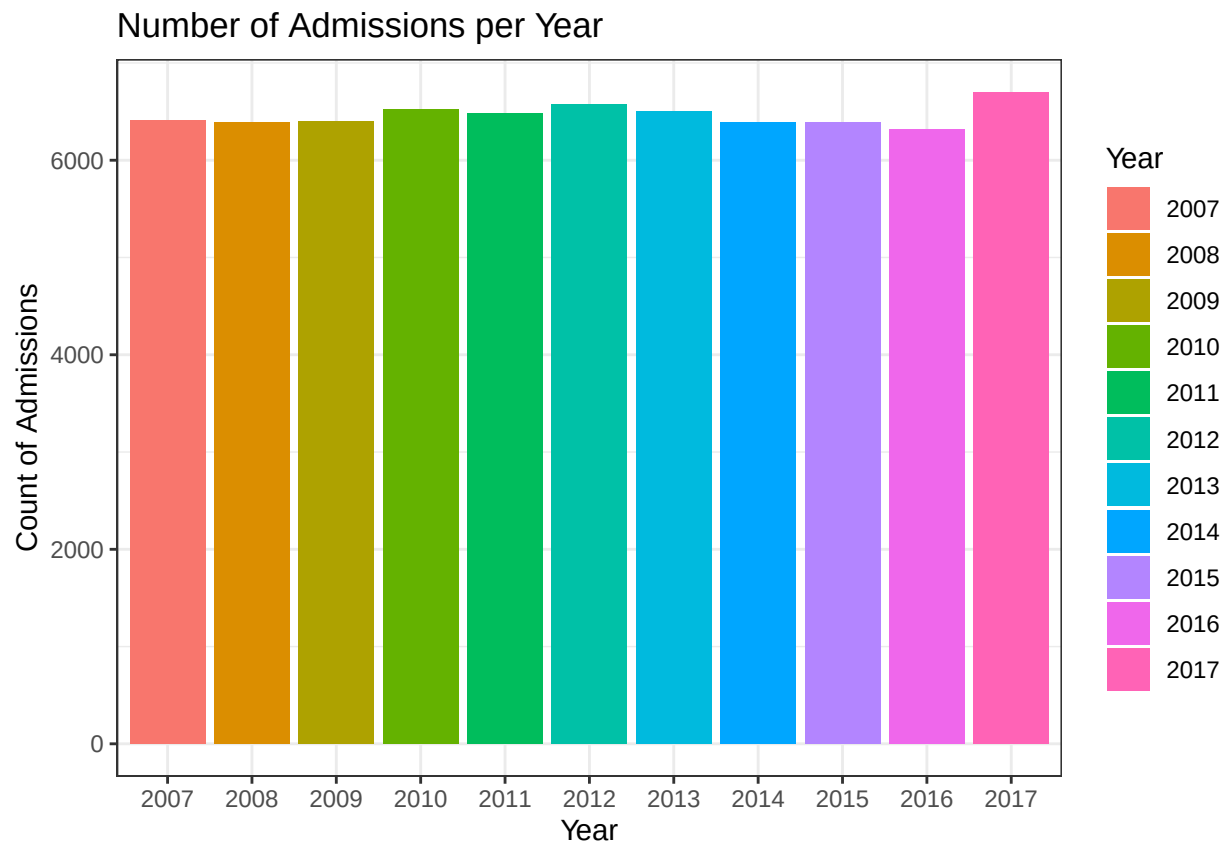
```
##   Year County Program_Category      Service_Type Age_Group  
## 1 2017 Albany      Crisis Medically Managed Detoxification 18 thru 24  
## 2 2017 Albany      Crisis Medically Managed Detoxification 18 thru 24  
## 3 2017 Albany      Crisis Medically Managed Detoxification 18 thru 24
```

```
## 4 2017 Albany Crisis Medically Managed Detoxification 18 thru 24
## 5 2017 Albany Crisis Medically Managed Detoxification 18 thru 24
## 6 2017 Albany Crisis Medically Managed Detoxification 25 thru 34
## Primary_Substance_Group Admissions
## 1 Alcohol 25
## 2 All Others 7
## 3 Cocaine incl Crack 1
## 4 Heroin 64
## 5 Other Opioids 20
## 6 Alcohol 140
```

STEP 5

Plot visualization:

```
ggplot(ny_cd_df_clean, aes(x = Year, fill = Year)) +
  geom_bar() +
  theme_bw() +
  labs(title = "Number of Admissions per Year",
       x = "Year",
       y = "Count of Admissions",
       fill = "Year")
```



The graph is a bar chart representing the Number of Admissions per Year from 2007 to 2017. Each bar

corresponds to a specific year and is color-coded according to the legend on the right. The x-axis represents the years, while the y-axis represents the count of admissions.

Question 3

Tibble 1: county which contains the name of all counties and their respective county code (link)[Links to an external site.](#) (which is the primary key). ensure that your data frame does not contain duplicate counties and ensure that your dataframe contains all counties in the data. The county codes provided in the link above are intended as a starting point. Remember that primary keys must be unique.

Data Cleaning and Transformation - Extracting County Codes In this part of the analysis, we are focusing on cleaning and transforming the data for better analysis. Specifically, we are working with the County column in the dataset to:

1. Extract unique counties
2. Generate a standardized code for each county
3. Display the cleaned data for verification

```
# Extract Unique Counties with County Codes
county <- ny_cd_df_clean %>%
  select(County) %>%
  distinct() %>%
  mutate(county_code = toupper(substr(County, 1, 2))) # Generate county code using first 2 letters

# Display cleaned county data
head(county)
```

```
##      County county_code
## 1    Albany          AL
## 2     Bronx          BR
## 3   Broome          BR
## 4 Dutchess          DU
## 5      Erie          ER
## 6    Kings          KI
```

We want to ensure that each county appears only once in our data.

1. We selected the County column from the original dataset, which contains multiple entries for the same county.
2. By using the `distinct()` function, we removed duplicates, leaving us with a list of unique counties.
3. `substr(County, 1, 2)`: We took the first two letters of each county's name. This is useful for creating short and unique identifiers.
4. `toupper()`: We converted these two letters to uppercase for consistency and standardization across the dataset.
5. The `head(county)` function displays the first few rows of our cleaned dataset.
6. The result should show a list of counties paired with their corresponding county codes

Tibble 2: program_category: which contains a unique identifier and the name of the program category. ensure that your data frame does not contain duplicates. The codes can be numeric (e.g. auto incremented).

Data Cleaning for Program Categories -

In this part of the analysis, we focus on cleaning and transforming the Program Category column for better analysis. We perform the following steps:

1. Extract unique program categories
2. Generate a standardized code for each program category
3. Display the cleaned data for verification

```
# Extract Unique Program Categories
program_category <- ny_cd_df_clean %>%
  select(Program_Category) %>%
  distinct() %>%
  mutate(program_code = toupper(substr(Program_Category, 1, 2))) # Assign unique numeric code

# Display cleaned program_category data
head(program_category)
```

```
##           Program_Category program_code
## 1              Crisis             CR
## 2             Inpatient             IN
## 3 Opioid Treatment Program           OP
## 4             Outpatient            OU
## 5             Residential            RE
```

We want to ensure that each program category appears only once in the dataset.

1. We selected the Program_Category column from the original dataset, which contains multiple entries for the same category.
2. By using the distinct() function, we removed duplicates, leaving only unique program categories.
3. substr(Program_Category, 1, 2): We extracted the first two letters of each program category. This gives us a concise, standardized code for each category.
4. toupper(): We converted these letters to uppercase to ensure consistency across the dataset.
5. The head(program_category) function displays the first few rows of the cleaned dataset.

Tibble 3: primary_substance_group: which contains a unique identifier and the name of the substance. ensure that your data frame does not contain duplicates. The codes can be numeric (e.g. auto incremented).

Data Cleaning and Transformation for Primary Substance Group -

In this section of the analysis, we are cleaning and transforming the Primary Substance Group data to:

1. Extract unique substance categories
2. Generate a standardized code for each substance group (e.g., “Heroin” becomes “H”)
3. Display the cleaned data with substance codes for verification.

```
primary_substance_group <- ny_cd_df_clean %>%
  select(Primary_Substance_Group) %>%
  distinct() %>%
  mutate(
    substance_code = case_when(
      Primary_Substance_Group == "All Others" ~ "AO", # Assign code 'AO' to "All Others"
      Primary_Substance_Group == "Heroin" ~ "H",      # Assign code 'H' to "Heroin"
      Primary_Substance_Group == "Cocaine incl Crack" ~ "CC", # Assign code 'C' to "Cocaine incl Crack"
      Primary_Substance_Group == "Alcohol" ~ "A",      # Assign code 'A' to "Alcohol"
```

```

    Primary_Substance_Group == "Other Opioids" ~ "O", # Assign code 'O' to "Other Opioids"
    Primary_Substance_Group == "Marijuana incl Hashish" ~ "MH", # Assign code 'M' to "Marijuana incl
  )
)

# Display cleaned primary_substance_group data with substance codes
head(primary_substance_group)

##   Primary_Substance_Group substance_code
## 1           Alcohol              A
## 2           All Others             AO
## 3   Cocaine incl Crack             CC
## 4           Heroin                H
## 5           Other Opioids          O
## 6 Marijuana incl Hashish          MH

```

We want to eliminate any duplicates and work with only the unique substance categories in the dataset.

1. We selected the Primary_Substance_Group column from the dataset.
2. The distinct() function removed any duplicate entries, leaving only unique substance categories.

We want to generate unique substance codes that will make the data easier to work with. 1. We used the case_when() function to create the substance_code column. Each substance group was assigned a corresponding code:

“All Others” → “AO” “Heroin” → “H” “Cocaine incl Crack” → “CC” “Alcohol” → “A” “Other Opioids” → “O” “Marijuana incl Hashish” → “MH”

To verify that the data transformation was performed correctly.

1. The head(primary_substance_group) function displays the first few rows of the cleaned dataset.

Tibble 4: admissions_data which contain the details on the reported number of admissions — excluding the data that resides in the county, program_category and primary_substance_group tibbles/data frames; you should instead include a column with their respective foreign keys.

Clean, merge, and transform admissions_data - The goal of this step is to clean, merge, and transform data for a more structured and effective analysis.

```

admissions_data <- ny_cd_df_clean %>%
  left_join(county, by = "County") %>%
  left_join(program_category, by = "Program_Category") %>%
  left_join(primary_substance_group, by = "Primary_Substance_Group")

# Display the final merged data
admissions_data <- admissions_data%>%
  select(Year, county_code, program_code, Service_Type, Age_Group, substance_code, Admissions)

#Conversion to a factor

admissions_data$county_code <- as.factor(admissions_data$county_code)
admissions_data$program_code <- as.factor(admissions_data$program_code)

```

```
admissions_data$substance_code <- as.factor(admissions_data$substance_code)

head(admissions_data)
```

```
##   Year county_code program_code      Service_Type Age_Group
## 1 2017          AL          CR Medically Managed Detoxification 18 thru 24
## 2 2017          AL          CR Medically Managed Detoxification 18 thru 24
## 3 2017          AL          CR Medically Managed Detoxification 18 thru 24
## 4 2017          AL          CR Medically Managed Detoxification 18 thru 24
## 5 2017          AL          CR Medically Managed Detoxification 18 thru 24
## 6 2017          AL          CR Medically Managed Detoxification 25 thru 34
##   substance_code Admissions
## 1              A           25
## 2             AO            7
## 3             CC            1
## 4              H           64
## 5              O           20
## 6              A          140
```

```
tail(admissions_data)
```

```
##   Year county_code program_code      Service_Type Age_Group
## 71078 2007          WE          RE Res Rehab for Youth 55 and Older
## 71079 2007          WE          RE Res Rehab for Youth   Under 18
## 71080 2007          WE          RE Res Rehab for Youth   Under 18
## 71081 2007          WE          RE Res Rehab for Youth   Under 18
## 71082 2007          WE          RE Res Rehab for Youth   Under 18
## 71083 2007          WE          RE Res Rehab for Youth   Under 18
##   substance_code Admissions
## 71078          CC            1
## 71079          A             6
## 71080          AO            1
## 71081          CC            4
## 71082          H             1
## 71083          MH          236
```

We are merging multiple datasets to combine relevant data into a single dataframe, `admissions_data`, for better analysis.

Joining the datasets using common columns:

1. County: We join by the County column.
2. Program Category: We join by Program_Category.
3. Primary Substance Group: We join by Primary_Substance_Group.

Once the data is merged, we keep only the relevant columns for our analysis:

1. Year
2. County Code
3. Program Code
4. Service Type
5. Age Group

6. Substance Code
7. Admissions

This helps reduce unnecessary data and focus on key variables.

We convert categorical columns (such as county_code, program_code, and substance_code) to factors.

After performing the necessary transformations, we check the data to ensure everything is in order.

We use the following functions to display the first and last rows of the data:

1. head(admissions_data): View the first few rows.
2. tail(admissions_data): View the last few rows.

This ensures that the data has been correctly merged and transformed.

Question 4

Create a function called annualAdmissions() that derives the total number of reported admissions that transpired each year, for the entire state of NY and displays the results using a line chart. Annotate the chart to show the year with the highest number of admissions. Note: the year should be on the x-axis and the number of admissions on the y-axis. Explain the chart.

```
# Define the function
annualAdmissions <- function(ny_cd_df_clean) {

  # Aggregate total admissions by year
  admissions_by_year <- ny_cd_df_clean %>%
    group_by(Year) %>%
    summarise(total_admissions = sum(Admissions, na.rm = TRUE))%>%
    arrange(Year)

  # Identify the year with the highest admissions
  max_year <- admissions_by_year %>%
    filter(total_admissions == max(total_admissions))

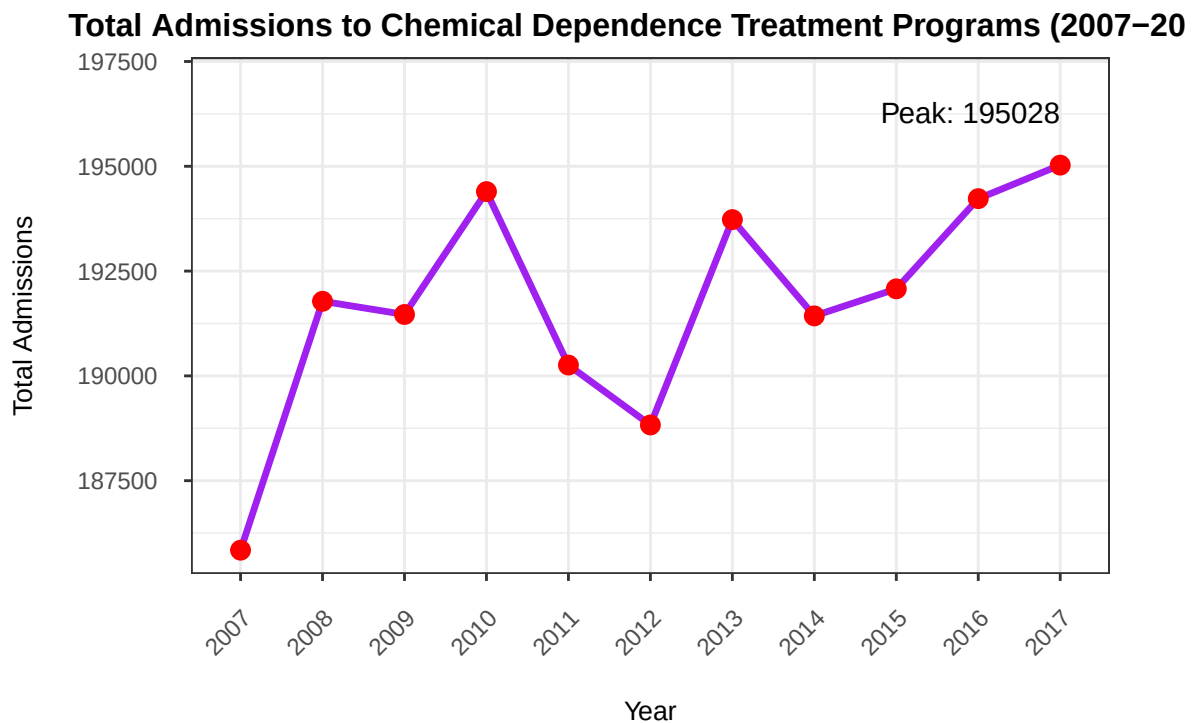
  # Create the line chart
  ggplot(admissions_by_year, aes(x = factor(Year), y = total_admissions)) +
    geom_line(group = 1, color = "purple", size = 1.2) +
    geom_point(color = "red", size = 3) +
    geom_text(data = max_year, aes(label = paste("Peak:", total_admissions)),
              vjust = -2, hjust = 1, size = 4, color = "black") +
    #scale_x_continuous(breaks = seq(2007, 2019, by = 1)) +
    scale_y_continuous(limits = c(min(admissions_by_year$total_admissions),
                                   max(admissions_by_year$total_admissions) + 2000)) +
    labs(title = "Total Admissions to Chemical Dependence Treatment Programs (2007-2017)",
         x = "Year",
         y = "Total Admissions") +
    theme_bw() +
    theme(
      plot.title = element_text(hjust = 0.5, size = 12, face = "bold"),
      axis.title = element_text(size = 10),
      axis.text.x = element_text(angle = 45, hjust = 1, margin = margin(t = 10)),
      plot.margin = margin(1, 1, 1, 1, "cm"),
      axis.text.y = element_text(margin = margin(r = 10)),
      axis.title.x = element_text(margin = margin(t = 15)),
```



```
axis.title.y = element_text(margin = margin(r = 15))
)
}
```

```
# Run the function with admissions_data
annualAdmissions(ny_cd_df_clean)
```

```
## Warning: Using `size` aesthetic for lines was deprecated in ggplot2 3.4.0.
## i Please use `linewidth` instead.
## This warning is displayed once every 8 hours.
## Call `lifecycle::last_lifecycle_warnings()` to see where this warning was
## generated.
```



The line chart “Total Admissions to Chemical Dependence Treatment Programs (2007-2017)” presents the trend of total admissions to chemical dependence treatment programs over the years 2007 to 2019.

General Trend: The number of admissions fluctuates over the years with periodic increases and decreases rather than a steady trend. Despite fluctuations, there appears to be an overall upward trend in total admissions towards the later years.

Peaks & Dips: Highest Admissions: The most recent year (likely 2017) shows the highest admissions, indicating an increasing demand for treatment programs. Significant Drop: Admissions declined around 2011-2012 before rising again.

Color Representation: The purple line represents the trend of admissions. Red points highlight specific yearly data points, making fluctuations more visible.

X-Axis & Y-Axis: The X-axis (Years) shows the timeline from 2007 to 2019. The Y-axis (Total Admissions) represents the number of admissions, suggesting that values range between 185,000 to 197,500.

Question 5

Analyze the percentage of admissions for each county and visualize the results for the top 5 counties using a bar chart. Explain the results. Note: ensure that you join any related dataframes/ tibbles.

```
# Load necessary libraries
library(dplyr)
library(ggplot2)

# Defining the function
countyAdmissionsAnalysis <- function(admissions_data) {

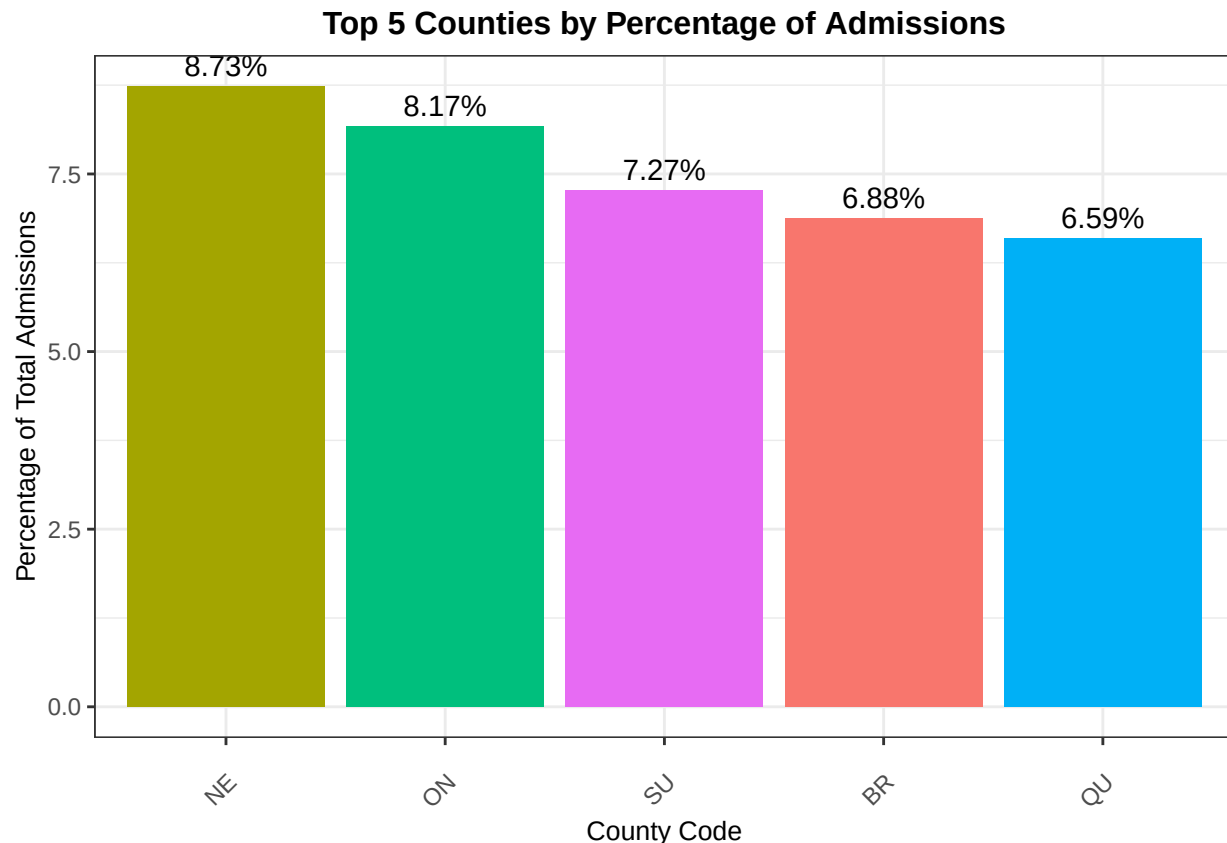
  # separate total admissions by county
  county_admissions <- admissions_data %>%
    group_by(county_code) %>%
    summarise(total_admissions = sum(Admissions, na.rm = TRUE)) %>%
    arrange(desc(total_admissions))

  # Calculating percentage of total admissions for each county
  total_statewide_admissions <- sum(county_admissions$total_admissions, na.rm = TRUE)
  county_admissions <- county_admissions %>%
    mutate(percentage = (total_admissions / total_statewide_admissions) * 100)

  # Selecting top 5 counties
  top_5_counties <- county_admissions %>%
    slice_max(order_by = total_admissions, n = 5)

  # bar chart
  ggplot(top_5_counties, aes(x = reorder(county_code, -total_admissions), y = percentage, fill = county_code)) +
    geom_bar(stat = "identity") +
    geom_text(aes(label = paste0(round(percentage, 2), "%")), vjust = -0.5, size = 4) +
    labs(title = "Top 5 Counties by Percentage of Admissions",
         x = "County Code",
         y = "Percentage of Total Admissions") +
    theme_bw() +
    theme(
      plot.title = element_text(hjust = 0.5, size = 12, face = "bold"),
      axis.title = element_text(size = 10),
      axis.text.x = element_text(angle = 45, hjust = 1, margin = margin(t = 10)),
      legend.position = "none"
    )
}

# Run the function with admissions_data
countyAdmissionsAnalysis(admissions_data)
```



The bar chart represents **the top 5 counties in terms of percentage of total admissions to chemical dependence treatment programs.**

From the graph we can see that the county **NE** has highest percentage of total admission, this means that higher population, greater availability of treatment facilities.

The second highest is **ON** , followed by **SU**, **BR**, **QU**, these are the counties that have treatment program admissions.

The reason for high admission is that irban aread have higher substance use rates which in-turn has high accessible treatment programs.

In bar chart color-coded bars help differentiate counties, and the percentage on top of each bars is to make visualization easy to compare across the counties.

In conclusion, this visualization helps identify the counties which has the highest treatment admissions and the county with highest percentage of admissions is **NE**. While **NE** is highest, the difference between top counties are not huge, which says that the admissions are somewhat spread out among them. This information is important because the health officials and policy makers could know where to allocate resources and arrange treatment programs which in-turn helps those struggling with substance dependence.

Question 6

Filter the data, using a regular expression, and extract all admissions to the various “Rehab” facilities; i.e. your regex should match all facilities that include the word rehab, rehabilitation, etc. Using the filtered data, identify which substance is the most prominent among each age group. Visualize and explain the results.

The provided R code analyzes admissions to rehabilitation facilities based on substance use and age groups. First, it filters the dataset to include only rehabilitation-related services by applying a regular expression that matches keywords like “rehab” and “rehabilitation” in the Service_Type column.

The filtered dataset is then grouped by Age_Group and substance_code, summing up the admissions to determine the total count for each substance within each age category.

To identify the most prominent substance for each age group, the code selects the substance with the highest admissions using slice_max(), ensuring only one is chosen per group. The results are visualized using a bar chart, where the x-axis represents age groups, the y-axis represents total admissions, and different colors represent the most commonly used substance in each group.

The visualization shows that heroin (H) is the most common substance among the 18-24 age group, while alcohol (A) is the leading substance for all older age groups, with the highest admissions occurring in the 45-54 age range.

Additionally, among individuals under 18, mental health-related substances (MH) are the most reported. This pattern suggests that younger adults are more affected by heroin use, while alcohol addiction becomes more prominent with increasing age. The results highlight significant trends in substance abuse treatment, indicating the need for age-specific intervention strategies in rehabilitation services.

```
# Define the function
rehabAdmissionsAnalysis <- function(ny_cd_df_clean) {

  # Filter admissions related to rehab facilities using regex on Service_Type
  rehab_admissions <- ny_cd_df_clean %>%
    filter(str_detect(Service_Type, regex("rehab|rehabilitation", ignore_case = TRUE)))

  # Print the first few rows of rehab_admissions
  print(head(rehab_admissions)) # Displays the first 6 rows

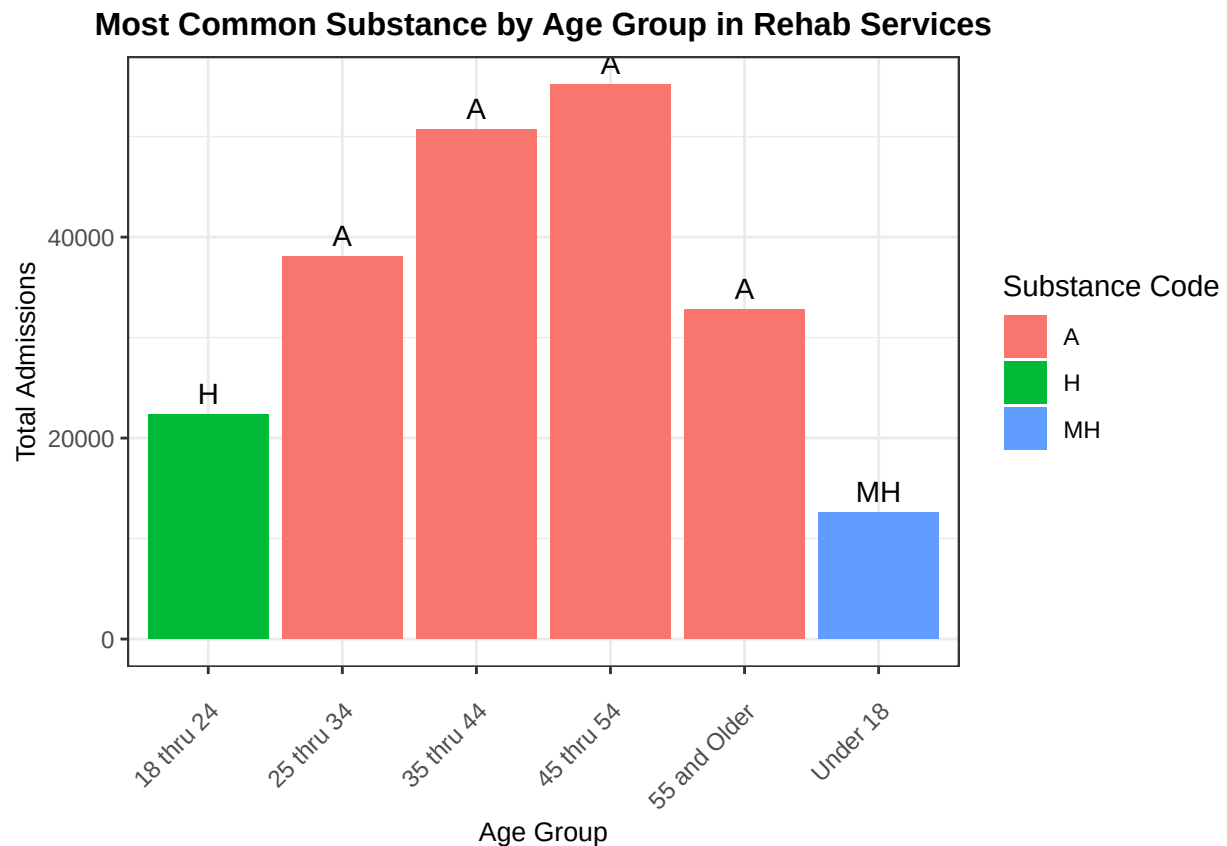
  # Identify the most common substance for each age group
  top_substance_by_age <- rehab_admissions %>%
    group_by(Age_Group, substance_code) %>%
    summarise(total_admissions = sum(Admissions, na.rm = TRUE), .groups = "drop") %>%
    group_by(Age_Group) %>%
    slice_max(order_by = total_admissions, n = 1, with_ties = FALSE) # Get the top substance for each age group

  # Bar chart visualization for all age groups
  ggplot(top_substance_by_age, aes(x = Age_Group, y = total_admissions, fill = substance_code)) +
    geom_bar(stat = "identity", position = "dodge") + # Use "dodge" to separate bars for each age group
    geom_text(aes(label = substance_code), vjust = -0.5, size = 4, position = position_dodge(width = 0.5))
  labs(title = "Most Common Substance by Age Group in Rehab Services",
       x = "Age Group",
       y = "Total Admissions",
       fill = "Substance Code") +
  theme_bw() +
  theme(
    plot.title = element_text(hjust = 0.5, size = 12, face = "bold"),
    axis.title = element_text(size = 10),
    axis.text.x = element_text(angle = 45, hjust = 1, margin = margin(t = 10))
  )
}

# Run the function
rehabAdmissionsAnalysis(admissions_data)
```

```
##      Year county_code program_code      Service_Type  Age_Group
```

## 1	2017	AL	IN Inpatient Rehabilitation 18 thru 24
## 2	2017	AL	IN Inpatient Rehabilitation 18 thru 24
## 3	2017	AL	IN Inpatient Rehabilitation 18 thru 24
## 4	2017	AL	IN Inpatient Rehabilitation 18 thru 24
## 5	2017	AL	IN Inpatient Rehabilitation 18 thru 24
## 6	2017	AL	IN Inpatient Rehabilitation 18 thru 24
##	substance_code	Admissions	
## 1	A	11	
## 2	AO	2	
## 3	CC	4	
## 4	H	21	
## 5	MH	6	
## 6	O	5	



The bar chart shows the most common substance for each age group in rehab services. In 18-24 age groups, the most common substance is H (Heroin) (green bar).

In 25+ age groups, the most common substance is A (likely Alcohol) (red bars), peaking in the 45-54 age group.

And in Under 18, the most common substance is MH (blue bar).

These results suggest that younger individuals (18-24) are more affected by heroin use, while older individuals struggle more with alcohol addiction.